



**MSc. Computer Engineering, Cybersecurity
and Artificial Intelligence**

Fraud Detection using Random Forest

Prepared by
Mohamed NJAH

Table of contents

I.	Introduction	5
II.	Motivation and state of the art	6
III.	Dataset	6
IV.	Methodology	7
1.	Data Preprocessing	8
a.	Merging Transaction and Identity	8
b.	Removing columns with high number of missing values	8
c.	Handling the remaining missing values	9
d.	Encoding categorical columns	9
e.	Under sampling	10
2.	Random Forest Training	11
3.	Hyperparameter tuning	12
4.	Cross Validation	13
5.	Model Evaluation	13
V.	Results	15
1.	Decision Tree Baseline	15
2.	Random Forest Hyperparameter Tuning	15
3.	Final Model Performance	17
VI.	Conclusion	18

I. Introduction

The global transition toward digital payments has transformed commerce, according to the latest joint report by the European Banking Authority (EBA) and the European Central Bank (ECB), payment fraud across the European Economic Area (EEA) reached €4.2 billion in 2024, up from €3.5 billion in 2023¹.

Historically, financial institutions relied on rule-based systems, simple heuristics such as "flag transactions over \$500" or "block purchases from high-risk countries." While these approaches caught obvious fraud, they suffered from critical limitations:

- **High false positive rates:** for every actual fraud case, rule-based systems flag many legitimate transactions, frustrating customers and costing merchants sales.
- **Inability to adapt:** fraudsters continuously evolve their tactics, but rule-based systems require manual updates, creating persistent windows of vulnerability.
- **Missed complex patterns:** sophisticated fraud schemes involve subtle correlations across multiple variables that simple rules cannot capture.

These limitations have driven the financial industry toward machine learning. Unlike static rules, ML models can identify complex, non-linear patterns, adapt automatically to emerging fraud trends, and reduce false positives while maintaining high detection accuracy.

This report will study the applicability of the Random Forest algorithm to solve fraud detection using the IEEE-CIS Fraud Detection dataset.

¹ [Joint EBA-ECB report on payment fraud](#)

II. Motivation and state of the art

The growing sophistication of fraudulent transactions demands detection systems that go beyond simple rule-based heuristics. Machine learning offers a pathway to automatically learn complex patterns from historical data, adapt to evolving fraud strategies, and flag suspicious activities in real time.

Several previous studies have demonstrated the effectiveness of Random Forest for fraud detection. For instance, Bhattacharyya et al. (2011)² compared Random Forest with support vector machines and logistic regression on a real-world credit card dataset, concluding that Random Forest achieved superior predictive performance while maintaining reasonable computational efficiency. More recently, Xuan et al. (2018)³ applied a Random Forest model to a Chinese bank's transaction data, reporting high accuracy and low false positive rates, and noting the algorithm's resilience to class imbalance when combined with resampling techniques.

Despite these successes, the application of Random Forest to fraud detection is not without challenges. The severe class imbalance (fraud often accounts for less than 5% of transactions) can bias the model toward the majority class, necessitating careful handling through resampling techniques.

III. Dataset

The IEEE-CIS Fraud Detection dataset⁴, released for a Kaggle competition by IEEE in collaboration with Vesta, is designed to mirror the complexity of real-world

² Bhattacharyya, S., Jha, S., Tharakunnel, K., & Westland, J. C. (2011). Data mining for credit card fraud: A comparative study. *Decision Support Systems*, 50(3), 602–613.

³ Xuan, S., Liu, G., Li, Z., Zheng, L., Wang, S., & Jiang, C. (2018). Random forest for credit card fraud detection. In 2018 IEEE 15th International Conference on Networking, Sensing and Control (ICNSC), 1–6.

⁴ [IEEE-CIS Fraud Detection Dataset](#)

financial transactions. It consists of two tables, transaction and identity, linked by a unique **TransactionID**. Not all transactions have corresponding identity information, reflecting the incomplete data often encountered in production environments.

The transaction table contains 394 features describing the payment event itself. These include TransactionAMT (payment amount in USD), TransactionDT (a timedelta from a reference datetime, not an actual timestamp for privacy), and ProductCD (product code). Payment card information (card1 to card6), address details (addr1, addr2, dist), email domains (P_emaildomain, R_emaildomain), and various masked features such as counting variables (C1 to C14), timedeltas (D1 to D15), and match indicators (M1 to M9) are also provided. The Vxxx features are rich, anonymised variables engineered by Vesta's fraud detection system, including rankings, counts, and entity relations.

The identity table captures the digital context of each transaction, including network information (IP, ISP, proxy) and digital signature details (browser, operating system, device model). These features (id_12–id_38, DeviceType, DeviceInfo) are categorical and masked to protect privacy and honor contractual agreements with data partners.

The data is provided in four files: train_transaction.csv, train_identity.csv, test_transaction.csv, and test_identity.csv. The training set includes the binary target isFraud (1 for fraudulent, 0 otherwise), while the test set requires participants to predict fraud probabilities. The dataset is severely imbalanced, reflecting the rarity of fraud in real payment systems.

IV. Methodology

Our methodology comprises several stages, from data preprocessing to model training and evaluation. The following steps outline our approach:

1. Data Preprocessing

a. Merging Transaction and Identity

To create a unified dataset for modeling, Transaction and Identity tables were merged on the common key TransactionID. A right join was performed for the training set to ensure all transactions are retained even if they lack corresponding identity information, the same approach was applied to the test set.

```
#merge transaction and identity to have only one table for each train and test
train = pd.merge(train_id,train_trans,left_on='TransactionID',right_on='TransactionID',how='right')
test = pd.merge(test_id,test_trans,left_on='TransactionID',right_on='TransactionID',how='right')
```

Figure 1: Merging transaction and Identity

This resulted in a single training dataframe and a single test dataframe, combining all available features.

b. Removing columns with high number of missing values

Many columns in the dataset contained a large proportion of missing values, which can introduce noise and reduce model performance. The percentage of missing values was computed for each column in the training set, and any column with more than 80% missing data was removed from both the training and test sets.

```
# percentage of missing values per column
missing_pct = train.isnull().mean() * 100
# columns with more than 80% missing values
cols_to_drop = missing_pct[missing_pct > 80].index
# drop those columns
train = train.drop(columns=cols_to_drop)
test = test.drop(columns=cols_to_drop, errors='ignore')
```

Figure 2: Removing columns with high number of missing values

This threshold was an estimate chosen to eliminate features that are unlikely to provide useful signal while preserving as much information as possible.

c. Handling the remaining missing values

The remaining missing values were imputed. For categorical columns, the mode (most frequent value) from the training set was used to fill missing entries in both training and test sets. For numerical columns, the mean of the training set was employed. This approach ensures that the test set is processed using statistics derived only from the training data, avoiding data leakage.

```
# For categorical columns in both train and test
for col in train.select_dtypes(include=['object']).columns:
    # Fill train
    mode_val = train[col].mode()[0] if not train[col].mode().empty else 'Unknown'
    train[col] = train[col].fillna(mode_val)

    # Fill test with same mode
    if col in test.columns:
        test[col] = test[col].fillna(mode_val)

# For numerical columns in both train and test
for col in train.select_dtypes(exclude=['object']).columns:
    # Fill train
    mean_val = train[col].mean()
    train[col] = train[col].fillna(mean_val)

    # Fill test with same mean
    if col in test.columns:
        test[col] = test[col].fillna(mean_val)
```

Figure 3: Handling the remaining missing values

d. Encoding categorical columns

Scikit-learn's RandomForestClassifier is implemented to work with numerical inputs only, so all categorical features were transformed into numeric representations. A LabelEncoder was fitted on each categorical column of the training set, converting the unique string values into integers. The same encoders were then applied to the test set, any previously unseen category in the test set was assigned a value of -1 to indicate novelty.

```
cat_col=[]
for col in train:
    if train[col].dtype == 'object':
        cat_col.append(col)
encoders = {}
for col in cat_col:
    le = LabelEncoder()
    train[col] = le.fit_transform(train[col].astype(str).values)
    encoders[col] = le

for col in cat_col:
    if col in test.columns:
        test[col] = test[col].astype(str)
        # Create array for transformation
        encoded_values = []
        for val in test[col]:
            if val in encoders[col].classes_:
                encoded_values.append(encoders[col].transform([val])[0])
            else:
                encoded_values.append(-1)
        test[col] = encoded_values
```

Figure 4: Encoding categorical columns

This encoding preserves the categorical nature while making the data compatible with tree-based models.

e. Under sampling

To mitigate the imbalance of our dataset and prevent the model from being biased toward the majority class, random under-sampling was applied, the non-fraud class was randomly subsampled to match the number of fraud samples in the training set, creating a balanced training set.

Undersampling was applied exclusively to the training portion of the dataset. This maintains the natural class imbalance in the test set, enabling a realistic evaluation of model performance on data that mirrors real-world conditions.

```
train_x, train_y = RandomUnderSampler().fit_resample(train_x, train_y)
train_y.value_counts().plot.bar()
```

Figure 5: Undersampling

Undersampling was preferred over oversampling for two main reasons: it reduces the training set size, leading to faster model training, and it avoids the risk of overfitting that can occur when synthetic minority samples are generated. Since tree-based models like Random Forest are relatively robust to data reduction, undersampling provides an effective and computationally efficient way to address class imbalance without introducing artificial data points.

2. Random Forest Training

Random Forest is an ensemble learning method that constructs a multitude of decision trees at training time and outputs the class that is the mode of the individual tree's predictions. It introduces randomness in two ways: each tree is trained on a random set of samples of the original data, and at each node, only a random subset of features is considered for splitting. This decorrelation among trees reduces variance and helps prevent overfitting.

The implementation used in this project is the RandomForestClassifier from the scikit-learn library⁵. This implementation builds each decision tree using the Classification and Regression Trees algorithm (CART), which recursively partitions the feature space with binary splits. The splitting criterion can be either Gini impurity or entropy, in our experiments we used entropy to measure split quality.

⁵ [Official documentation of the RandomForestClassifier of Scikit-learn](#)

Unlike other algorithms such as ID3, which can produce multi-way splits and handle only categorical features, CART produces binary trees and naturally accommodates both numerical and categorical data (after appropriate encoding). The scikit-learn implementation also provides extensive hyperparameters for controlling tree depth, minimum samples per split, and the number of features considered at each node, which we will experiment with.

3. Hyperparameter tuning

GridSearchCV was used to perform an exhaustive search over a predefined hyperparameter grid **using cross-validation** on top to identify the optimal model configuration. For each parameter combination, it evaluates performance using cross-validated ROC AUC and selects the set that yields the best average score, ensuring that the chosen parameters generalize well rather than overfitting to a single validation split.

```
# Hyperparameter tuning with GridSearchCV
param_grid = {
    'n_estimators': [100, 200, 400],
    'max_depth': [None, 10, 20, 40],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 5]
}

rf_grid = RandomForestClassifier(random_state=50, criterion='entropy')
grid_search = GridSearchCV(rf_grid, param_grid, cv=5, scoring='roc_auc', n_jobs=-1, verbose=0)
grid_search.fit(X_tr, y_tr)
```

Figure 6: Hyperparameter tuning using GridSearchCV

The following Random Forest hyperparameters were tuned:

- **n_estimators**: Number of trees in the forest.
- **max_depth**: Maximum depth of each tree.
- **min_samples_split**: Minimum samples required to split an internal node.
- **min_samples_leaf**: Minimum samples required at a leaf node.

Testing with various values of hyperparameters will serve useful for understanding each parameter and ensuring the best possible training results.

4. Cross Validation

StratifiedKFold was used as a cross-validation technique that preserves the class distribution across all folds. This ensures each fold maintains the same proportion of fraudulent and non-fraudulent transactions as the original dataset, providing stable and reliable performance estimates for imbalanced classification problems.

```
# Define StratifiedKFold for cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Figure 7: Cross validation using StratifiedKFold

StratifiedKFold was used right after GridSearchCV to apply cross validation on the best hyperparameter set and further evaluate the model using the dedicated evaluation metrics.

5. Model Evaluation

The following metrics were computed from the cross-validated predictions and probabilities to assess the performance of the best Random Forest model:

- **ROC AUC (Receiver Operating Characteristic Area Under the Curve):**
Measures the model's ability to discriminate between fraudulent and non-fraudulent transactions across all possible classification thresholds. It is computed as the probability that a randomly chosen fraud case receives a higher predicted probability than a randomly chosen non-fraud case.

$$\text{ROC AUC} = \int_0^1 \text{TPR}(t) d(\text{FPR}(t))$$

With:

$$\text{TPR} = \frac{TP}{TP + FN}, \text{FPR} = \frac{FP}{FP + TN}$$

where TPR is the true positive rate (recall/sensitivity) and FPR is the false positive rate (specificity) at threshold t .

- **Accuracy:** The proportion of correctly classified transactions (both fraud and non-fraud). While intuitive, accuracy can be misleading in imbalanced settings.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

- **F1-Score:** The harmonic mean of precision and recall, providing a single balanced measure that considers both false positives and false negatives.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Confusion Matrix:** A table showing the counts of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), allowing detailed inspection of error types.

ROC AUC was chosen as the primary metric for hyperparameter tuning and model evaluation for several reasons:

- **Threshold independence:** In fraud detection, the final decision threshold can be adjusted based on business needs (e.g., acceptable false positive rate). ROC AUC evaluates the model's ranking quality across all thresholds, making it suitable before a specific threshold is chosen.
- **Robustness to class imbalance:** Unlike accuracy, which can be inflated by simply predicting the majority class, ROC AUC considers both true positive and false positive rates, providing a balanced view even when fraud is rare.
- **Comprehensive assessment:** By summarizing the trade-off between sensitivity and specificity, ROC AUC gives a holistic measure of the model's discriminative power, complementing the more operational metrics (precision, recall, F1) that are used at a fixed threshold.

V. Results

1. Decision Tree Baseline

A single decision tree was trained on the balanced training data. The mean ROC AUC achieved was **0.7955**, indicating moderate discriminative power.

```

=== Decision Tree (on validation set) ===
Accuracy: 0.7955480280667796
F1 Score: 0.7994779307071666
ROC AUC: 0.7955480280667796
Confusion Matrix:
[[3207  926]
 [ 764 3369]]

```

	precision	recall	f1-score	support
Not Fraud	0.81	0.78	0.79	4133
Fraud	0.78	0.82	0.80	4133
accuracy			0.80	8266
macro avg	0.80	0.80	0.80	8266
weighted avg	0.80	0.80	0.80	8266

Figure 8: Decision tree classification results

The decision tree, while simple and interpretable, is prone to overfitting and does not capture complex patterns as effectively as ensemble methods.

2. Random Forest Hyperparameter Tuning

Grid search with 5-fold cross-validation was performed over a hyperparameter space.

The best configuration achieved a mean ROC AUC of **0.9218 ± 0.0037** using:

n_estimators=400, max_depth=40, min_samples_split=2, and min_samples_leaf=1.

To gather more evaluation metrics, the best model was retrained using StratifiedKFold with 5 folds, it gave the results below:

```

=== Best Random Forest (5-fold CV) ===
ROC AUC scores: [0.92630603 0.92608969 0.92842252 0.92975043 0.92607765]
Mean ROC AUC: 0.9273 ± 0.0015
Accuracy: 0.8507961089870784
F1 Score: 0.8476402273288856
ROC AUC: 0.9273159866806966
Confusion Matrix:
[[18008  2655]
 [ 3511 17152]]

```

	precision	recall	f1-score	support
Not Fraud	0.84	0.87	0.85	20663
Fraud	0.87	0.83	0.85	20663
accuracy			0.85	41326
macro avg	0.85	0.85	0.85	41326
weighted avg	0.85	0.85	0.85	41326

Figure 9: Best random forest classification results

To understand the influence of each hyperparameter, we fixed the best values for three parameters and varied the fourth. The results are shown below:

- **Effect of n_estimators:**

n_estimators	100	200	400
ROC AUC	0.9196	0.9212	0.9218

We can interpret that increasing the number of trees improves performance, but gains diminish beyond 200.

- **Effect of max_depth**

max_depth	10	20	40	None
ROC AUC	0.8819	0.9141	0.9218	0.9218

Depth has the most pronounced impact, shallow trees perform poorly, while depths of 40 or unlimited yield near-identical results.

- **Effect of min_samples_split**

min_samples_split	2	5	10
ROC AUC	0.9218	0.9205	0.9177

Smaller split values allow finer decision boundaries and lead to better performance.

- **Effect of min_samples_leaf**

min_samples_leaf	1	5
ROC AUC	0.9218	0.9113

Increasing the minimum leaf size causes a drop in ROC AUC, indicating that fine-grained leaves are essential for capturing fraud patterns.

3. Final Model Performance

After identifying the optimal hyperparameters through grid search, the final Random Forest model was trained on the entire balanced training set (X_{train} , y_{train}). Since the true labels for the test set are withheld by IEEE and Kaggle, we could not compute performance metrics locally. Instead, we generated predicted fraud probabilities for the test transactions and submitted them to the Kaggle platform, where they were evaluated against the hidden ground truth using the official ROC AUC metric. The test set itself reflects realistic traffic conditions, with fraudulent

transactions comprising only a small fraction of the total, preserving the severe class imbalance inherent in real-world payment systems.

Our submission achieved a public leaderboard score of **0.9165** and a private leaderboard score of **0.8936**, indicating strong but not perfect generalization to unseen, imbalanced data. These results validate the effectiveness of the Random Forest classifier, combined with careful preprocessing and hyperparameter tuning, for detecting fraudulent transactions in a realistic setting.

Submission and Description		Private Score 	Public Score 	Selected
	submission.csv Complete (after deadline) · 5d ago	0.893667	0.916518	☐

Figure 10: Kaggle competition submission

VI. Conclusion

This study explored the application of a Random Forest classifier to the IEEE-CIS Fraud Detection dataset, with the goal of developing an effective and computationally efficient solution for identifying fraudulent credit card transactions. Through a structured preprocessing pipeline the dataset was transformed into a format suitable for tree-based learning.

The above results indicate that Random Forest, when combined with thoughtful preprocessing, serves as a lightweight yet powerful solution for fraud detection. Its ability to capture non-linear relationships, handle mixed data types, and provide feature importance insights makes it particularly well-suited for deployment in resource-constrained environments where interpretability and computational efficiency are valued.

Nevertheless, several models are known to outperform Random Forest on similar tasks. Gradient boosting machines (e.g., XGBoost, LightGBM, CatBoost) often achieve superior predictive accuracy by sequentially correcting errors of previous trees and handling missing data natively. Deep neural networks, particularly those

incorporating attention mechanisms or autoencoders for anomaly detection, have also shown promise in capturing subtle fraud patterns.

In conclusion, while Random Forest provides a strong baseline and a practical solution for fraud detection, ongoing advances in ensemble learning and deep architectures continue to push the boundaries of what is achievable. The choice of model should ultimately be guided by the specific requirements of the deployment environment, balancing accuracy, interpretability, and computational cost. This project demonstrates that even a single Random Forest, when carefully tuned and evaluated, can form the core of an effective fraud detection system.

Appendix 1: Preprocessing

Importing necessary packages

```
import pandas as pd
import gc
from sklearn.preprocessing import LabelEncoder
from imblearn.under_sampling import RandomUnderSampler
```

Reading raw data

```
# read dataset
train_trans = pd.read_csv('../train_transaction.csv')
train_id = pd.read_csv('../train_identity.csv')
test_trans = pd.read_csv('../test_transaction.csv')
test_id = pd.read_csv('../test_identity.csv')

#merge transaction and identity to have only one table for each train and test
train =
pd.merge(train_id,train_trans,left_on='TransactionID',right_on='TransactionID',how='right')
test =
pd.merge(test_id,test_trans,left_on='TransactionID',right_on='TransactionID',how='right')

# free up some memory
del train_id
del train_trans
del test_id
del test_trans
gc.collect()

0
```

Eliminating columns with a high number of missing values

```
# percentage of missing values per column
missing_pct = train.isnull().mean() * 100
# columns with more than 80% missing values
cols_to_drop = missing_pct[missing_pct > 80].index
# drop those columns
train = train.drop(columns=cols_to_drop)
test = test.drop(columns=cols_to_drop, errors='ignore')
```

Handling missing values

```
# For categorical columns in both train and test
for col in train.select_dtypes(include=['object']).columns:
    # Fill train
    mode_val = train[col].mode()[0] if not train[col].mode().empty
    else 'Unknown'
    train[col] = train[col].fillna(mode_val)

    # Fill test with same mode
    if col in test.columns:
        test[col] = test[col].fillna(mode_val)

# For numerical columns in both train and test
for col in train.select_dtypes(exclude=['object']).columns:
    # Fill train
    mean_val = train[col].mean()
    train[col] = train[col].fillna(mean_val)

    # Fill test with same mean
    if col in test.columns:
        test[col] = test[col].fillna(mean_val)

print("Missing values in train:", train.isnull().sum().sum())
print("Missing values in test:", test.isnull().sum().sum())
```

```
Missing values in train: 0
Missing values in test: 0
```

Encode categorical columns

```
cat_col=[]
for col in train:
    if train[col].dtype == 'object':
        cat_col.append(col)
encoders = {}
for col in cat_col:
    le = LabelEncoder()
    train[col] = le.fit_transform(train[col].astype(str).values)
    encoders[col] = le

for col in cat_col:
    if col in test.columns:
        test[col] = test[col].astype(str)
        # Create array for transformation
        encoded_values = []
        for val in test[col]:
            if val in encoders[col].classes_:
                encoded_values.append(encoders[col].transform([val]))
```

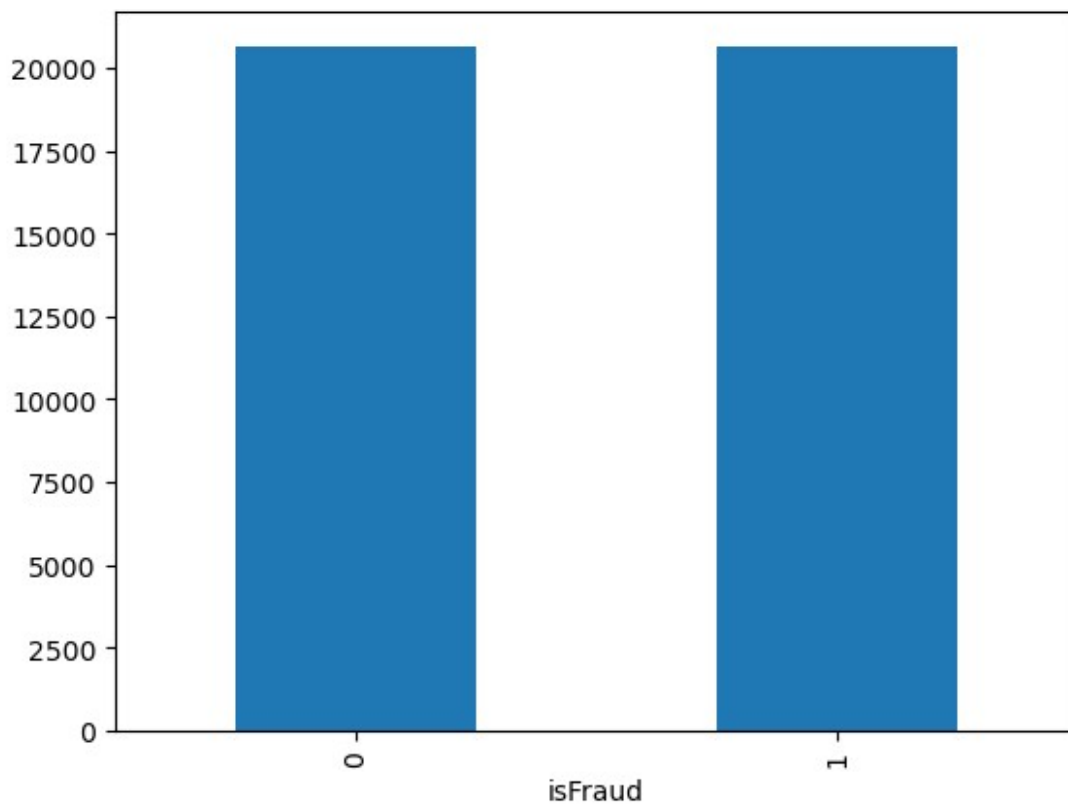
```
[0])
        else:
            encoded_values.append(-1)
        test[col] = encoded_values
gc.collect()
0

train_y = train['isFraud']
train_x = train.drop(columns=['isFraud'])
```

Undersampling

```
train_x, train_y = RandomUnderSampler().fit_resample(train_x, train_y)
train_y.value_counts().plot.bar()
```

<Axes: xlabel='isFraud'>



```
train_x.to_csv('../preprocessed dataset/X_train.csv', index=False)
test.to_csv('../preprocessed dataset/X_test.csv', index=False)
train_y.to_csv('../preprocessed dataset/y_train.csv', index=False)
```

Appendix 2: Classifying the preprocessed dataset

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score,
confusion_matrix, roc_auc_score, classification_report
from sklearn.model_selection import train_test_split, cross_val_score,
cross_val_predict, GridSearchCV, StratifiedKFold

# read dataset
X_test = pd.read_csv('../preprocessed dataset\X_test.csv')
X_train = pd.read_csv('../preprocessed dataset\X_train.csv')
y_train = pd.read_csv('../preprocessed dataset\y_train.csv')

# Convert y_train to 1D array from DataFrame
y_train = y_train.values.ravel()

# Keep TransactionID for later use in submission file
test_ids = X_test['TransactionID'].copy()
# Drop TransactionID from both sets for training
X_train = X_train.drop(columns=['TransactionID'], errors='ignore')
X_test = X_test.drop(columns=['TransactionID'], errors='ignore')

# Split training data into train and validation
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train,
test_size=0.2, random_state=42, stratify=y_train)

dt_model = DecisionTreeClassifier(random_state=1)
dt_model.fit(X_tr, y_tr)
dt_pred = dt_model.predict(X_val)
dt_proba = dt_model.predict_proba(X_val)[:, 1]

print("=== Decision Tree (on validation set) ===")
print("Accuracy:", accuracy_score(y_val, dt_pred))
print("F1 Score:", f1_score(y_val, dt_pred))
print("ROC AUC:", roc_auc_score(y_val, dt_proba))
print("Confusion Matrix:\n", confusion_matrix(y_val, dt_pred))
print(classification_report(y_val, dt_pred, target_names=['Not Fraud',
'Fraud']))

=== Decision Tree (on validation set) ===
Accuracy: 0.7955480280667796
F1 Score: 0.7994779307071666
ROC AUC: 0.7955480280667796
Confusion Matrix:
[[3207  926]
```

[764 3369]]				
	precision	recall	f1-score	support
Not Fraud	0.81	0.78	0.79	4133
Fraud	0.78	0.82	0.80	4133
accuracy			0.80	8266
macro avg	0.80	0.80	0.80	8266
weighted avg	0.80	0.80	0.80	8266

Basic Random Forest Classifier

```
# Basic Random Forest with default parameters
rf_default = RandomForestClassifier(criterion='entropy',
random_state=50, n_estimators=100)
rf_default.fit(X_tr, y_tr)
rf_pred = rf_default.predict(X_val)
rf_proba = rf_default.predict_proba(X_val)[: , 1]

print("\n=== Random Forest (default, on validation set) ===")
print("Accuracy:", accuracy_score(y_val, rf_pred))
print("F1 Score:", f1_score(y_val, rf_pred))
print("ROC AUC:", roc_auc_score(y_val, rf_proba))
print("Confusion Matrix:\n", confusion_matrix(y_val, rf_pred))
print(classification_report(y_val, rf_pred, target_names=['Not Fraud',
'Fraud']))
```

```
=== Random Forest (default, on validation set) ===
```

```
Accuracy: 0.8518025647229616
```

```
F1 Score: 0.8495270851246776
```

```
ROC AUC: 0.9302395682300503
```

```
Confusion Matrix:
```

```
[[3583  550]
```

```
[ 675 3458]]
```

	precision	recall	f1-score	support
Not Fraud	0.84	0.87	0.85	4133
Fraud	0.86	0.84	0.85	4133
accuracy			0.85	8266
macro avg	0.85	0.85	0.85	8266
weighted avg	0.85	0.85	0.85	8266

```
# Hyperparameter tuning with GridSearchCV
```

```
param_grid = {
    'n_estimators': [100, 200, 400],
    'max_depth': [None, 10, 20, 40],
```

```

    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 5]
}

rf_grid = RandomForestClassifier(random_state=50, criterion='entropy')
grid_search = GridSearchCV(rf_grid, param_grid, cv=5,
scoring='roc_auc', n_jobs=-1, verbose=0)
grid_search.fit(X_tr, y_tr)

# Display all results
results_df = pd.DataFrame(grid_search.cv_results_)

# Show relevant columns: parameters, mean test score, std test score
display_cols = ['params', 'mean_test_score', 'std_test_score']
results_df = results_df[display_cols].sort_values('mean_test_score',
ascending=False)
print("\n=== All Grid Search Results ===")
print(results_df.to_string(index=False))

# Define StratifiedKFold for cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Model evaluation
best_rf = grid_search.best_estimator_
y_pred_cv = cross_val_predict(best_rf, X_train, y_train, cv=cv,
method='predict')
y_proba_cv = cross_val_predict(best_rf, X_train, y_train, cv=cv,
method='predict_proba')[:, 1]
best_cv_scores = cross_val_score(best_rf, X_train, y_train, cv=cv,
scoring='roc_auc')
print("\n=== Best Random Forest (5-fold CV) ===")
print("ROC AUC scores:", best_cv_scores)
print("Mean ROC AUC: {:.4f} ± {:.4f}".format(best_cv_scores.mean(),
best_cv_scores.std()))
print("Accuracy:", accuracy_score(y_train, y_pred_cv))
print("F1 Score:", f1_score(y_train, y_pred_cv))
print("ROC AUC:", roc_auc_score(y_train, y_proba_cv))
print("Confusion Matrix:\n", confusion_matrix(y_train, y_pred_cv))
print(classification_report(y_train, y_pred_cv, target_names=['Not
Fraud', 'Fraud']))

=== All Grid Search Results ===

params  mean_test_score  std_test_score
{'max_depth': 40, 'min_samples_leaf': 1, 'min_samples_split': 2,
'n_estimators': 400}      0.921813      0.003745
{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2,
'n_estimators': 400}      0.921789      0.004168
{'max_depth': 40, 'min_samples_leaf': 1, 'min_samples_split': 2,

```

'n_estimators': 200}	0.921156	0.003762
{'max_depth': None, 'n_estimators': 200}	'min_samples_leaf': 1,	'min_samples_split': 2,
'n_estimators': 200}	0.920917	0.004051
{'max_depth': None, 'n_estimators': 400}	'min_samples_leaf': 1,	'min_samples_split': 5,
'n_estimators': 400}	0.920545	0.003923
{'max_depth': 40, 'n_estimators': 400}	'min_samples_leaf': 1,	'min_samples_split': 5,
'n_estimators': 400}	0.920543	0.004182
{'max_depth': 40, 'n_estimators': 200}	'min_samples_leaf': 1,	'min_samples_split': 5,
'n_estimators': 200}	0.920021	0.004029
{'max_depth': None, 'n_estimators': 200}	'min_samples_leaf': 1,	'min_samples_split': 5,
'n_estimators': 200}	0.919998	0.003972
{'max_depth': 40, 'n_estimators': 100}	'min_samples_leaf': 1,	'min_samples_split': 2,
'n_estimators': 100}	0.919612	0.003715
{'max_depth': None, 'n_estimators': 100}	'min_samples_leaf': 1,	'min_samples_split': 2,
'n_estimators': 100}	0.919516	0.004398
{'max_depth': 40, 'n_estimators': 100}	'min_samples_leaf': 1,	'min_samples_split': 5,
'n_estimators': 100}	0.918610	0.004219
{'max_depth': None, 'n_estimators': 100}	'min_samples_leaf': 1,	'min_samples_split': 5,
'n_estimators': 100}	0.918264	0.003642
{'max_depth': None, 'n_estimators': 400}	'min_samples_leaf': 1,	'min_samples_split': 10,
'n_estimators': 400}	0.917905	0.004455
{'max_depth': 40, 'n_estimators': 400}	'min_samples_leaf': 1,	'min_samples_split': 10,
'n_estimators': 400}	0.917699	0.004532
{'max_depth': 40, 'n_estimators': 200}	'min_samples_leaf': 1,	'min_samples_split': 10,
'n_estimators': 200}	0.917222	0.004482
{'max_depth': None, 'n_estimators': 200}	'min_samples_leaf': 1,	'min_samples_split': 10,
'n_estimators': 200}	0.917219	0.004569
{'max_depth': None, 'n_estimators': 100}	'min_samples_leaf': 1,	'min_samples_split': 10,
'n_estimators': 100}	0.916324	0.004725
{'max_depth': 40, 'n_estimators': 100}	'min_samples_leaf': 1,	'min_samples_split': 10,
'n_estimators': 100}	0.916108	0.004373
{'max_depth': 20, 'n_estimators': 400}	'min_samples_leaf': 1,	'min_samples_split': 2,
'n_estimators': 400}	0.914076	0.004903
{'max_depth': 20, 'n_estimators': 200}	'min_samples_leaf': 1,	'min_samples_split': 2,
'n_estimators': 200}	0.913435	0.004952
{'max_depth': 20, 'n_estimators': 400}	'min_samples_leaf': 1,	'min_samples_split': 5,
'n_estimators': 400}	0.913207	0.004608
{'max_depth': 20, 'n_estimators': 200}	'min_samples_leaf': 1,	'min_samples_split': 5,
'n_estimators': 200}	0.912625	0.004577
{'max_depth': 20, 'n_estimators': 100}	'min_samples_leaf': 1,	'min_samples_split': 2,
'n_estimators': 100}	0.912491	0.005318
{'max_depth': 20, 'n_estimators': 100}	'min_samples_leaf': 1,	'min_samples_split': 5,
'n_estimators': 100}	0.911751	0.004810
{'max_depth': 40, 'n_estimators': 400}	'min_samples_leaf': 5,	'min_samples_split': 2,
'n_estimators': 400}	0.911309	0.004647
{'max_depth': 40, 'n_estimators': 400}	'min_samples_leaf': 5,	'min_samples_split': 5,
'n_estimators': 400}	0.911309	0.004647
{'max_depth': 40, 'n_estimators': 400}	'min_samples_leaf': 5,	'min_samples_split': 10,
'n_estimators': 400}	0.911309	0.004647

```

{'max_depth': 20, 'min_samples_leaf': 1, 'min_samples_split': 10,
'n_estimators': 400}      0.911115      0.004658
{'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 5,
'n_estimators': 400}      0.911076      0.004544
{'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 2,
'n_estimators': 400}      0.911076      0.004544
{'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 10,
'n_estimators': 400}      0.911076      0.004544
{'max_depth': 40, 'min_samples_leaf': 5, 'min_samples_split': 10,
'n_estimators': 200}      0.910792      0.004588
{'max_depth': 40, 'min_samples_leaf': 5, 'min_samples_split': 2,
'n_estimators': 200}      0.910792      0.004588
{'max_depth': 40, 'min_samples_leaf': 5, 'min_samples_split': 5,
'n_estimators': 200}      0.910792      0.004588
{'max_depth': 20, 'min_samples_leaf': 1, 'min_samples_split': 10,
'n_estimators': 200}      0.910737      0.004824
{'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 5,
'n_estimators': 200}      0.910550      0.004502
{'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 10,
'n_estimators': 200}      0.910550      0.004502
{'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 2,
'n_estimators': 200}      0.910550      0.004502
{'max_depth': 20, 'min_samples_leaf': 1, 'min_samples_split': 10,
'n_estimators': 100}      0.910356      0.004953
{'max_depth': 40, 'min_samples_leaf': 5, 'min_samples_split': 5,
'n_estimators': 100}      0.909855      0.004754
{'max_depth': 40, 'min_samples_leaf': 5, 'min_samples_split': 10,
'n_estimators': 100}      0.909855      0.004754
{'max_depth': 40, 'min_samples_leaf': 5, 'min_samples_split': 2,
'n_estimators': 100}      0.909855      0.004754
{'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 2,
'n_estimators': 100}      0.909702      0.004346
{'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 10,
'n_estimators': 100}      0.909702      0.004346
{'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 5,
'n_estimators': 100}      0.909702      0.004346
{'max_depth': 20, 'min_samples_leaf': 5, 'min_samples_split': 5,
'n_estimators': 400}      0.906811      0.004922
{'max_depth': 20, 'min_samples_leaf': 5, 'min_samples_split': 10,
'n_estimators': 400}      0.906811      0.004922
{'max_depth': 20, 'min_samples_leaf': 5, 'min_samples_split': 2,
'n_estimators': 400}      0.906811      0.004922
{'max_depth': 20, 'min_samples_leaf': 5, 'min_samples_split': 10,
'n_estimators': 200}      0.906156      0.005056
{'max_depth': 20, 'min_samples_leaf': 5, 'min_samples_split': 5,
'n_estimators': 200}      0.906156      0.005056
{'max_depth': 20, 'min_samples_leaf': 5, 'min_samples_split': 2,
'n_estimators': 200}      0.906156      0.005056
{'max_depth': 20, 'min_samples_leaf': 5, 'min_samples_split': 5,

```

'n_estimators': 100}	0.905500	0.004915
{'max_depth': 20, 'min_samples_leaf': 5, 'min_samples_split': 2,		
'n_estimators': 100}	0.905500	0.004915
{'max_depth': 20, 'min_samples_leaf': 5, 'min_samples_split': 10,		
'n_estimators': 100}	0.905500	0.004915
{'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2,		
'n_estimators': 400}	0.881850	0.005807
{'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 5,		
'n_estimators': 400}	0.881538	0.005862
{'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2,		
'n_estimators': 200}	0.881402	0.005734
{'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 10,		
'n_estimators': 400}	0.881335	0.005918
{'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2,		
'n_estimators': 100}	0.881096	0.005591
{'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 5,		
'n_estimators': 200}	0.881026	0.005765
{'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 10,		
'n_estimators': 100}	0.881015	0.005798
{'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 10,		
'n_estimators': 200}	0.880987	0.005952
{'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 5,		
'n_estimators': 100}	0.880763	0.005523
{'max_depth': 10, 'min_samples_leaf': 5, 'min_samples_split': 2,		
'n_estimators': 400}	0.880583	0.005998
{'max_depth': 10, 'min_samples_leaf': 5, 'min_samples_split': 5,		
'n_estimators': 400}	0.880583	0.005998
{'max_depth': 10, 'min_samples_leaf': 5, 'min_samples_split': 10,		
'n_estimators': 400}	0.880583	0.005998
{'max_depth': 10, 'min_samples_leaf': 5, 'min_samples_split': 5,		
'n_estimators': 200}	0.879995	0.005939
{'max_depth': 10, 'min_samples_leaf': 5, 'min_samples_split': 2,		
'n_estimators': 200}	0.879995	0.005939
{'max_depth': 10, 'min_samples_leaf': 5, 'min_samples_split': 10,		
'n_estimators': 200}	0.879995	0.005939
{'max_depth': 10, 'min_samples_leaf': 5, 'min_samples_split': 10,		
'n_estimators': 100}	0.879957	0.005784
{'max_depth': 10, 'min_samples_leaf': 5, 'min_samples_split': 2,		
'n_estimators': 100}	0.879957	0.005784
{'max_depth': 10, 'min_samples_leaf': 5, 'min_samples_split': 5,		
'n_estimators': 100}	0.879957	0.005784

=== Best Random Forest (5-fold CV) ===

ROC AUC scores: [0.92630603 0.92608969 0.92842252 0.92975043
0.92607765]

Mean ROC AUC: 0.9273 ± 0.0015

Accuracy: 0.8507961089870784

F1 Score: 0.8476402273288856

ROC AUC: 0.9273159866806966

Confusion Matrix:

```
[[18008 2655]
 [ 3511 17152]]
```

	precision	recall	f1-score	support
Not Fraud	0.84	0.87	0.85	20663
Fraud	0.87	0.83	0.85	20663
accuracy			0.85	41326
macro avg	0.85	0.85	0.85	41326
weighted avg	0.85	0.85	0.85	41326

Train the best model on the full training data

```
final_model = grid_search.best_estimator_  
final_model.fit(X_train, y_train)
```

Predict probabilities on the test set

```
test_proba = final_model.predict_proba(X_test)[:, 1]
```

Create submission DataFrame

```
if test_ids is not None:  
    submission = pd.DataFrame({'TransactionID': test_ids, 'isFraud':  
test_proba})  
else:  
    submission = pd.DataFrame({'isFraud': test_proba})  
    submission.index.name = 'TransactionID'
```

```
submission.to_csv('submission.csv', index=False)  
print("\nSubmission file saved as submission.csv")
```

Submission file saved as submission.csv